

Duoc UC

Escuela de Informática y Telecomunicaciones

BookPoint Chile

Informe Técnico – Arquitectura de Microservicios

Evaluación Final Transversal – Situación Evaluativa 1: Entrega por Encargo

Desarrollo Fullstack I (DSY1103)

Estudiante: [Nombre del estudiante]

Docente: [Nombre del docente]

15 de julio de 2026

Índice general

1. Introducción	3
1.1. Contexto	3
1.2. Objetivo del informe	3
1.3. Alcance del proyecto	3
2. Estrategia y Arquitectura de Microservicios	5
2.1. Análisis del sistema actual	5
2.2. Estrategia de microservicios elegida	5
2.3. Diseño de los microservicios	6
2.3.1. Alcance y exclusiones deliberadas	6
2.4. Plan de migración	7
3. Evaluación de Enfoques Éticos	8
3.1. Privacidad de datos de clientes	8
3.2. Responsabilidad en el despliegue	8
3.3. Impacto en el empleo	9
3.4. Desafíos éticos específicos enfrentados durante el desarrollo	9
4. Buenas Prácticas de Desarrollo Backend	11
4.1. Patrón Controller-Service-Repository	11
4.2. Modularidad, mantenibilidad y escalabilidad	12
4.2.1. Base de datos por servicio	12
4.2.2. Separación DTO / entidad	12
4.2.3. Manejo centralizado de excepciones	13
4.3. Spring Boot y Maven	13
4.4. Trabajo colaborativo con Git	14
4.5. Evolución del proyecto: correcciones realizadas durante el semestre	15
5. Operaciones CRUD con JPA/ORM validadas con Postman	17
5.1. Persistencia con Spring Data JPA	17
5.2. Validación con la colección Postman	18
5.3. Reportes de ventas	18
6. Comunicación RESTful Entre Microservicios	20
6.1. Feign Client como mecanismo de comunicación	20
6.2. Manejo de errores en comunicación remota	21
6.3. Validación con Postman	21

7. Pruebas Unitarias	22
7.1. Herramientas utilizadas	22
7.2. Estrategia de pruebas	22
7.3. Resultados de cobertura	23
8. Documentación HATEOAS	25
8.1. Framework elegido: Spring HATEOAS	25
8.2. Navegabilidad de la API	25
9. Conclusión	27

1. Introducción

1.1. Contexto

BookPoint Chile es una empresa nacional dedicada a la venta de libros, artículos de papelería y material educativo. Nació como una librería independiente en Concepción y, debido al crecimiento sostenido de sus ventas, abrió nuevas sucursales en Temuco y La Serena.

Actualmente, la empresa opera con un sistema monolítico que concentra ventas, inventario, despacho y atención de clientes en un único aplicativo. El aumento sostenido de pedidos, tanto presenciales como por canal web, ha comenzado a provocar lentitud en los procesos, problemas de stock desactualizado entre sucursales y demoras en la gestión de despachos.

1.2. Objetivo del informe

Este informe documenta la propuesta de solución tecnológica desarrollada para modernizar el sistema de BookPoint Chile, migrándolo desde su arquitectura monolítica actual hacia una arquitectura de microservicios independientes. En concreto, cubre los siete aspectos exigidos por la pauta de la Evaluación Final Transversal para la Situación Evaluativa 1 (Entrega por Encargo):

1. La estrategia de microservicios utilizada para resolver el caso propuesto.
2. La evaluación de los enfoques éticos considerados en su implementación.
3. Las buenas prácticas de diseño, arquitectura, y las herramientas (Spring, Maven, Git) utilizadas para desarrollar los componentes de backend.
4. La implementación de operaciones CRUD con JPA/ORM, validadas con Postman.
5. La comunicación RESTful entre microservicios, validada con Postman.
6. El desarrollo de pruebas unitarias con JUnit.
7. La implementación de un framework de documentación HATEOAS.

1.3. Alcance del proyecto

La solución desarrollada consta de **diez microservicios de negocio** (catálogo, inventario, sucursales, clientes, carrito, pedidos, pagos, envíos, notificaciones y reseñas), además

de un servidor de descubrimiento (Eureka) y un API Gateway como punto de entrada único.

Se priorizó cubrir en profundidad el **flujo comercial completo** descrito en el caso – desde que un cliente explora el catálogo hasta que recibe su pedido y puede dejar una reseña –, con validaciones reales entre servicios, manejo de errores consistente y pruebas automatizadas. Quedan deliberadamente fuera del alcance actual funciones administrativas de soporte descritas en el caso (autenticación y roles, reportes de venta, promociones, facturación electrónica, transferencias entre sucursales), que se detallan junto con su justificación en la sección [2](#).

2. Estrategia y Arquitectura de Microservicios

2.1. Análisis del sistema actual

Según el caso de estudio, BookPoint Chile nació como una librería independiente en Concepción y, debido al crecimiento de sus ventas, abrió nuevas sucursales en Temuco y La Serena. Actualmente opera con un **sistema monolítico** que concentra en un mismo aplicativo las ventas, el inventario, el despacho y la atención de clientes.

El propio caso identifica tres síntomas concretos de que ese monolito ya no escala con el crecimiento de la operación:

- **Lentitud en los procesos**, provocada por el aumento sostenido de pedidos tanto presenciales como por canal web.
- **Stock desactualizado** entre sucursales, al no existir una fuente de verdad de inventario que se sincronice en tiempo real entre Concepción, Temuco y La Serena.
- **Demoras en la gestión de despachos**, al no contar con un módulo dedicado de logística que pueda evolucionar independientemente del resto del sistema.

Estos síntomas son característicos de una arquitectura monolítica sometida a crecimiento: al compartir un único proceso, una única base de datos y un único ciclo de despliegue, cualquier cambio en el módulo de inventario obliga a redespargar *todo* el sistema – incluida la caja de venta presencial, que no puede detenerse sin afectar directamente la operación de las tres tiendas físicas. Del mismo modo, un pico de tráfico en el canal web (por ejemplo, durante una campaña de descuentos) satura recursos que también necesita la venta presencial, porque ambos comparten el mismo proceso.

2.2. Estrategia de microservicios elegida

Para resolver el caso propuesto se optó por una arquitectura de **microservicios independientes**, organizados por dominio de negocio (no por capa técnica), con los siguientes principios:

- **Base de datos por servicio**. Cada microservicio posee su propia base de datos MySQL (Tabla 4.1), eliminando el acoplamiento de datos que hoy sufre el sistema monolítico. El módulo de inventario puede evolucionar su modelo de stock sin coordinar una migración con el módulo de reseñas.

- **Comunicación síncrona vía REST/Feign** para las relaciones entre dominios que sí necesitan datos de otro servicio en el momento (por ejemplo, un pedido necesita confirmar que el libro existe y que hay stock antes de concretarse).
- **Descubrimiento de servicios con Eureka**, para que ningún microservicio necesite conocer de antemano la dirección de red de los demás – relevante en un escenario con tres sucursales que podrían eventualmente desplegarse en ubicaciones distintas.
- **API Gateway como fachada única**, que permite exponer una sola puerta de entrada hacia los distintos canales (presencial, web) sin que el cliente necesite conocer la topología interna de microservicios.

Esta estrategia ataca directamente los tres síntomas identificados: la lentitud se resuelve porque cada dominio escala y se despliega de forma independiente (el catálogo, que se consulta constantemente, no compite por recursos con el módulo de pagos); el stock desactualizado se resuelve centralizando el inventario en un único microservicio consultado por todos los canales y sucursales; y las demoras de despacho se resuelven con un microservicio de envíos dedicado, capaz de evolucionar su lógica de logística sin tocar el resto del sistema.

2.3. Diseño de los microservicios

El caso de estudio describe seis perfiles de usuario (Administrador del Sistema, Jefe de Sucursal, Asistente de Ventas, Área de Logística, Encargado de Bodega y Clientes vía web) con un total de más de treinta acciones posibles. La Tabla 2.1 mapea los diez microservicios efectivamente construidos a los perfiles del caso que cubren.

Microservicio	Perfiles/acciones del caso que cubre
Catálogo	Cliente: navegar catálogo de libros.
Inventario	Encargado de Bodega: administrar stock central y por sucursal. Administrador: consultar stock disponible en otras sucursales.
Sucursales	Jefe de Sucursal: administrar datos de la sucursal.
Clientes	Cliente: crear cuenta, administrar perfil y direcciones.
Carrito	Cliente: agregar productos al carrito.
Pedidos	Cliente: realizar compras online. Asistente de Ventas: registrar ventas (equivalente digital de la venta en caja). Jefe de Sucursal: generar reportes de ventas por sucursal y por autor.
Pagos	Cliente: pagar una compra online.
Envíos	Área de Logística: gestionar envíos de pedidos a domicilio, actualizar estados de pedidos.
Notificaciones	Cliente: recibir confirmaciones y alertas asociadas a su pedido.
Reseñas	Cliente: dejar reseñas y calificaciones de productos.

Cuadro 2.1: Mapeo de los microservicios construidos a los perfiles del caso BookPoint Chile

2.3.1. Alcance y exclusiones deliberadas

No todas las acciones descritas en el caso están cubiertas por la implementación actual. Se priorizó el **flujo comercial completo** (explorar catálogo, agregar al carrito, comprar,

pagar, despachar, reseñar) por sobre funciones administrativas de soporte, que quedaron fuera de alcance de forma consciente:

- Gestión de usuarios y permisos por rol para el perfil Administrador del Sistema (autenticación básica de clientes vía JWT sí se implementó, ver sección 3).
- Reportes de ventas por categoría (Jefe de Sucursal) – los reportes por sucursal y por autor sí se implementaron (sección 5), pero por categoría exigiría antes agregar un campo de categoría/género al catálogo, que no existe hoy.
- Promociones, descuentos, convenios estudiantiles y cupones (Asistente de Ventas, Cliente).
- Emisión de boletas o facturas electrónicas y gestión de devoluciones (Asistente de Ventas).
- Transferencias de stock entre sucursales, gestión de proveedores editoriales y optimización de rutas de distribución (Área de Logística, Encargado de Bodega).

Esta decisión de alcance no es un descuido: dado el tiempo disponible, se priorizó profundizar el flujo transaccional principal – con validaciones cruzadas reales entre servicios, manejo de errores consistente y pruebas automatizadas – por sobre cubrir superficialmente la totalidad de acciones descritas en el caso.

2.4. Plan de migración

Para migrar el sistema monolítico descrito en el caso hacia esta arquitectura, se plantea un enfoque incremental de tipo **strangler fig** (extracción progresiva), en lugar de una reescritura total (*big-bang rewrite*), que sería significativamente más riesgosa para una empresa que necesita mantener sus tres sucursales operativas durante la transición:

1. **Extraer primero los dominios de solo lectura y bajo acoplamiento transaccional:** catálogo e inventario. Son los más consultados (cada venta, presencial o web, necesita leerlos) y los que menos dependen de otros módulos para funcionar.
2. **Introducir el API Gateway como fachada de transición**, de modo que el resto del sistema (aún monolítico en esta etapa) y los microservicios ya extraídos convivan detrás de una única puerta de entrada, sin que el cliente note el cambio.
3. **Extraer los dominios transaccionales:** clientes, carrito, pedidos y pagos, en ese orden, ya que cada uno depende de que el anterior esté disponible como servicio (el carrito necesita clientes y catálogo; los pedidos necesitan catálogo e inventario).
4. **Extraer los dominios de soporte:** sucursales, envíos, notificaciones y reseñas, que dependen de que pedidos ya exista como microservicio independiente.
5. **Retirar el monolito** una vez que todos los dominios estén servidos por microservicios y validados en producción.

Este orden minimiza el riesgo operativo: en cada paso, el sistema sigue siendo funcional para las tres sucursales, y solo se apaga definitivamente el monolito cuando ya no queda ningún flujo de negocio dependiendo de él.

3. Evaluación de Enfoques Éticos

3.1. Privacidad de datos de clientes

El microservicio de clientes almacena datos personales identificables: nombre, correo electrónico, dirección y comuna. Al tratarse de un sistema que atiende tanto canal presencial como online, estos datos circulan entre varios microservicios (por ejemplo, **envíos** necesita la dirección de destino asociada a un pedido).

Frente a esto se aplicó un principio de **minimización de datos**: cada microservicio conoce únicamente los campos estrictamente necesarios para su función. El microservicio de pagos, por ejemplo, no tiene acceso al nombre ni a la dirección del cliente – solo al identificador del pedido y al monto a cobrar –, de modo que un eventual incidente de seguridad en el módulo de pagos no expondría por sí solo datos personales identificables.

Inicialmente el sistema no implementaba autenticación ni cifrado de credenciales, decisión de alcance consciente dado el tiempo disponible del proyecto y no un descuido. Sin embargo, precisamente por tratarse de datos personales de clientes – direcciones, historial de compras –, se decidió cerrar esa brecha al menos en el punto donde más importa: el microservicio de clientes ahora exige un token JWT válido para modificar o eliminar un perfil, y las contraseñas se almacenan con `BCryptPasswordEncoder` (hash con salt, nunca en texto plano). La protección se limitó deliberadamente a las operaciones de escritura sobre el propio perfil (`PUT/DELETE`) y no se extendió a las lecturas ni al resto de los microservicios: **carrito** y **reseñas** consultan clientes vía Feign para sus propias validaciones cruzadas, y esas llamadas de servicio a servicio no llevan el token de ningún usuario. Proteger también las lecturas habría exigido propagar un token de servicio a través de Feign – una solución más completa, pero fuera del alcance de tiempo disponible. Esta es, en sí misma, una decisión ética explícita: priorizar cerrar la brecha más sensible (que cualquiera pueda modificar los datos de otro cliente) por sobre una implementación completa pero más costosa.

3.2. Responsabilidad en el despliegue

BookPoint Chile opera simultáneamente en tres sucursales físicas y un canal web. A diferencia de un negocio puramente digital, una caída del sistema no solo afecta las ventas online: si el monolito actual falla, se detiene también la venta presencial en Concepción, Temuco y La Serena, porque todo depende del mismo proceso.

La arquitectura de microservicios propuesta mitiga parcialmente este riesgo mediante **aislamiento de fallas**: si el microservicio de reseñas deja de responder, el cliente puede

seguir explorando el catálogo, agregando productos al carrito y pagando su compra sin verse afectado. Esto traslada la responsabilidad del despliegue desde "no puede fallar nunca" (inviabile en cualquier sistema real) hacia "una falla puntual no debe paralizar el negocio completo" – un estándar más realista y, a la vez, más exigente en el diseño de cada servicio individual, porque cada uno debe manejar adecuadamente los casos en que sus dependencias no respondan (ver sección 6, manejo de errores en comunicación remota).

3.3. Impacto en el empleo

El sistema automatiza tareas hoy realizadas manualmente por el personal de BookPoint: el registro de ventas en caja (equivalente al microservicio de pedidos), la consulta de disponibilidad de stock entre sucursales (inventario) y el seguimiento del estado de un envío (envíos, con su historial de estados).

Es importante encuadrar este punto con honestidad: el sistema no reemplaza al Asistente de Ventas, al Encargado de Bodega ni al Área de Logística – automatiza tareas repetitivas y propensas a error humano (como mantener sincronizado el stock entre tres sucursales a mano), liberando tiempo para tareas que sí requieren juicio humano: atención al cliente, resolución de casos excepcionales, gestión de proveedores. El riesgo ético real no está en automatizar estas tareas puntuales, sino en cómo se gestione la transición del personal hacia esas nuevas responsabilidades – una decisión organizacional de BookPoint Chile que excede el alcance técnico de este informe, pero que vale la pena nombrar explícitamente como parte de la evaluación ética del proyecto.

3.4. Desafíos éticos específicos enfrentados durante el desarrollo

Más allá de los principios generales discutidos arriba, el desarrollo del proyecto presentó tensiones concretas entre avanzar rápido y hacerlo bien, y al menos una duda genuina sobre si una decisión de alcance era aceptable para un sistema real.

Errores que exigieron parar y corregir en vez de seguir avanzando. En varios momentos del desarrollo surgieron problemas que hubiera sido más rápido ignorar o parchar superficialmente, pero que se decidió corregir de raíz:

- Durante la mayor parte del desarrollo, cualquier recurso no encontrado devolvía 500 en lugar de 404 en los diez servicios. Era un error silencioso – el sistema "funcionaba" en el sentido de que no se caía –, pero devolver el código HTTP correcto es justamente lo que permite que un cliente de la API distinga un error suyo de un error del servidor. Se optó por parar y corregirlo en los diez servicios a la vez, en lugar de dejarlo para "después".
- Un cliente de la API podía enviar un id propio al crear un recurso, y ese id terminaba sobrescribiendo silenciosamente un recurso existente en vez de crear uno nuevo. Es el tipo de bug que puede pasar desapercibido en las demos (todo parece funcionar) pero que en un sistema real significaría corromper datos de otro cliente sin ningún aviso.

- Al centralizar la documentación Swagger de los diez microservicios a través del Gateway, las rutas no llegaban correctamente: hubo que re-enrutar manualmente los puertos en el YAML del Gateway y ajustar la configuración de cada microservicio para que calzara con lo que el Gateway esperaba. Y al containerizar los servicios, Swagger dejó de funcionar porque cada servicio reportaba su propia IP interna de Docker como servidor, en vez de una URL que el navegador del cliente pudiera alcanzar. Ninguno de los dos problemas era obvio a primera vista, y ambos se podrían haber dejado "así no más" con la excusa de que la API seguía respondiendo – se optó por diagnosticar la causa real en vez de silenciar el síntoma.
- La decisión de alcance de la autenticación JWT (sección 3) es en sí misma un ejemplo de esta tensión: la alternativa más rápida habría sido proteger todo el microservicio de clientes sin pensar en las consecuencias; se optó por invertir tiempo en entender qué llamadas dependían de un acceso sin token (`carrito` y `reseñas` vía Feign) antes de decidir el alcance correcto de la protección.

Duda genuina sobre decisiones de alcance. La ausencia inicial de autenticación fue la que generó más incomodidad real durante el desarrollo: saber que, mientras no se implementara, cualquiera con acceso a la API podía consultar o modificar los datos de cualquier cliente sin restricción, se sentía como una brecha real y no solo teórica – razón por la cual se decidió cerrarla al menos parcialmente antes de la entrega final, en lugar de dejarla únicamente documentada como limitación.

Una duda similar surgió con la exclusión de reportes de venta y de la gestión de roles administrativos (sección 2): un Jefe de Sucursal o un Administrador del Sistema reales necesitarían esas funciones para operar el negocio día a día, y su ausencia no es solo una limitación técnica, sino una función de negocio real que el caso pide. A diferencia de la autenticación, esta duda llevó a revisar el alcance antes de la entrega final: los reportes de ventas por sucursal y por autor se implementaron (sección 5), por ser acotados y no requerir cambios de arquitectura. La gestión de usuarios y permisos por rol, en cambio, se decidió dejar fuera deliberadamente – a diferencia de un reporte, que es una vista de solo lectura sobre datos que ya existen, un sistema de roles reales implica un modelo de autorización completo y una decisión arquitectónica (¿es un nuevo microservicio? ¿se integra a uno existente?) que merece más tiempo de análisis del que quedaba disponible, y apurarla habría sido peor que documentarla honestamente como limitación.

4. Buenas Prácticas de Desarrollo Backend

Este capítulo aborda el indicador de mayor ponderación del informe (16 %): el desarrollo de componentes de backend aplicando buenas prácticas de diseño y arquitectura, el uso de Spring y Maven, y el trabajo colaborativo mediante Git.

4.1. Patrón Controller-Service-Repository

Los diez microservicios de BookPoint aplican de forma consistente el patrón **Controller-Service-Repository (CSR)**, separando estrictamente tres responsabilidades:

- **Controller:** recibe la petición HTTP, delega en el service y traduce el resultado a una respuesta REST (código de estado, cuerpo, enlaces HATEOAS). No contiene lógica de negocio.
- **Service:** concentra la lógica de negocio, las validaciones cruzadas contra otros microservicios (vía Feign) y las reglas del dominio (por ejemplo, verificar stock antes de confirmar un pedido).
- **Repository:** acceso a datos mediante Spring Data JPA (`JpaRepository`), sin lógica de negocio.

El siguiente ejemplo, tomado del microservicio de catálogo, muestra la capa de servicio completa de `LibroServiceImpl`: no contiene código HTTP ni SQL, solo la traducción entre el DTO de entrada y la entidad persistida, y el manejo del caso "no encontrado" mediante una excepción de negocio propia:

Listing 4.1: `LibroServiceImpl.java` – capa de servicio de catálogo

```
1 @Service
2 @RequiredArgsConstructor
3 public class LibroServiceImpl implements LibroService {
4
5     private final LibroRepository libroRepository;
6
7     @Override
8     public Libro guardarLibro(LibroDTO libroDTO) {
9         Libro libro = new Libro();
10        libro.setTitulo(libroDTO.getTitulo());
11        libro.setAutor(libroDTO.getAutor());
12        libro.setPrecio(libroDTO.getPrecio());
```

```

13     return libroRepository.save(libro);
14 }
15
16 @Override
17 public Libro obtenerPorId(Long id) {
18     return libroRepository.findById(id)
19         .orElseThrow(() -> new RecursoNoEncontradoException(
20             "Libro no encontrado con id: " + id));
21 }
22 }

```

Esta separación se replica de forma idéntica en los diez servicios, lo que permite que cualquier persona familiarizada con uno de ellos pueda orientarse rápidamente en los demás – una consecuencia directa de mantener una convención uniforme en todo el proyecto.

4.2. Modularidad, mantenibilidad y escalabilidad

4.2.1. Base de datos por servicio

Cada microservicio posee su propia base de datos MySQL, sin compartir esquemas ni tablas con otros servicios (Tabla 4.1). Esto evita el acoplamiento de datos característico de los monolitos – como el descrito en el caso BookPoint Chile, donde ventas, inventario, despacho y atención de clientes conviven en un mismo sistema – y permite que cada equipo pueda evolucionar el modelo de datos de su dominio sin coordinar migraciones con el resto.

Microservicio	Base de datos
catálogo	bookpoint_catálogo_db
inventario	bookpoint_inventario_db
pedidos	bookpoint_pedidos_db
clientes	bookpoint_clientes_db
sucursales	bookpoint_sucursales_db
carrito	bookpoint_carrito_db
pagos	bookpoint_pagos_db
envíos	bookpoint_envios_db
notificaciones	bookpoint_notificaciones_db
reseñas	bookpoint_resenas_db

Cuadro 4.1: Una base de datos independiente por microservicio

4.2.2. Separación DTO / entidad

En los servicios donde la entrada requiere validación estricta (por ejemplo, catálogo, inventario y pedidos), se utiliza un DTO anotado con *Bean Validation* (`@NotBlank`, `@NotNull`, `@Positive`) en lugar de exponer la entidad JPA directamente en el controlador. Esto evita que un cliente de la API pueda, por ejemplo, enviar un `id` propio al crear un recurso y sobrescribir accidentalmente uno existente – un error real detectado y corregido durante el desarrollo (ver sección 4.5).

4.2.3. Manejo centralizado de excepciones

Cada microservicio expone un `@RestControllerAdvice` propio (`GlobalExceptionHandler`) que traduce las excepciones de negocio en respuestas HTTP coherentes, evitando repetir bloques `try/catch` en cada controlador:

Listing 4.2: `GlobalExceptionHandler.java` (catálogo) – fragmento

```

1  @RestControllerAdvice
2  public class GlobalExceptionHandler {
3
4      @ExceptionHandler(RecursoNoEncontradoException.class)
5      public ResponseEntity<ErrorResponseDTO> handleRecursoNoEncontrado(
6          RecursoNoEncontradoException ex) {
7          ErrorResponseDTO errorDTO = new ErrorResponseDTO(
8              ex.getMessage(), HttpStatus.NOT_FOUND.value(),
9              LocalDateTime.now(), null);
10         return new ResponseEntity<>(errorDTO, HttpStatus.NOT_FOUND);
11     }
12
13     @ExceptionHandler(IllegalArgumentException.class)
14     public ResponseEntity<ErrorResponseDTO> handleArgumentoInvalido(
15         IllegalArgumentException ex) {
16         ErrorResponseDTO errorDTO = new ErrorResponseDTO(
17             ex.getMessage(), HttpStatus.BAD_REQUEST.value(),
18             LocalDateTime.now(), null);
19         return new ResponseEntity<>(errorDTO, HttpStatus.BAD_REQUEST);
20     }
21 }

```

Este patrón asegura que un recurso inexistente responda siempre con 404, una violación de regla de negocio con 400, y un error inesperado con 500 – de forma consistente en los diez servicios, en vez de depender de que cada desarrollador recuerde hacerlo caso a caso.

4.3. Spring Boot y Maven

El proyecto está organizado como un **POM multi-módulo** de Maven: un `pom.xml` raíz de tipo `pom` declara los doce módulos (diez microservicios de negocio, `eureka-server` y `gateway`) y centraliza en `dependencyManagement` las versiones compartidas – **Spring Boot 4.1**, **Java 21** y **Spring Cloud 2025.1.2** – de modo que ningún módulo hijo necesita fijar versiones por su cuenta:

Listing 4.3: `pom.xml` raíz – fragmento relevante

```

1  <properties>
2      <java.version>21</java.version>
3      <spring-cloud.version>2025.1.2</spring-cloud.version>
4  </properties>
5
6  <dependencyManagement>
7      <dependencies>
8          <dependency>

```

```

9      <groupId>org.springframework.cloud</groupId>
10     <artifactId>spring-cloud-dependencies</artifactId>
11     <version>${spring-cloud.version}</version>
12     <type>pom</type>
13     <scope>import</scope>
14 </dependency>
15 </dependencies>
16 </dependencyManagement>

```

Además de `spring-boot-starter-web` y `spring-boot-starter-data-jpa` (presentes en los diez servicios), el proyecto utiliza:

- **Spring Cloud OpenFeign:** comunicación declarativa entre microservicios (capítulo 6).
- **Spring Cloud Netflix Eureka Client:** registro y descubrimiento de servicios.
- **Spring HATEOAS:** navegabilidad de la API (capítulo 8).
- **springdoc-openapi:** documentación Swagger/OpenAPI por servicio.
- **JaCoCo Maven Plugin:** medición de cobertura de pruebas (capítulo 7).
- **Datafaker:** generación de datos de prueba realistas al levantar cada servicio (ver sección 4.5).

4.4. Trabajo colaborativo con Git

Este proyecto fue desarrollado íntegramente en solitario: aunque la pauta permite equipos de hasta tres integrantes, todo el desarrollo backend – diseño, código, pruebas y correcciones – fue trabajo individual, sin otros integrantes aportando código al repositorio.

El trabajo se hizo directo sobre la rama `main`, sin ramas separadas por funcionalidad. El desarrollo comenzó por los servicios núcleo del flujo comercial (catálogo, inventario y pedidos), contruidos hasta tener la comunicación Feign básica funcionando entre ellos; a partir de ahí se fue extendiendo el resto de los microservicios y, en tareas más mecánicas y repetitivas – aplicar el mismo cambio de forma consistente a lo largo de los diez servicios, como la incorporación de HATEOAS o la construcción de la colección Postman –, se aceleró el trabajo con herramientas de asistencia de desarrollo.

El repositorio del proyecto utiliza Git como sistema de control de versiones, con una convención de mensajes de commit en español, explicando siempre el *porqué* del cambio y no solo el *qué*. Los cambios no relacionados se separan en commits distintos en lugar de agruparse en una sola entrega masiva – por ejemplo, la sesión de trabajo dedicada a corregir el mapeo de errores HTTP, agregar HATEOAS y sumar validaciones cruzadas vía Feign se dividió en tres commits independientes, cada uno enfocado en un solo cambio lógico:

Listing 4.4: Extracto de `git log -oneline`

```

1 47bacbb feat: implementar HATEOAS en los 9 microservicios que faltaban
2 9c05bdb feat: agregar validaciones cruzadas via Feign en carrito,
3     resenas, pagos y envios

```

4.5. Evolución del proyecto: correcciones realizadas durante el semestre

A diferencia de las evaluaciones parciales anteriores – que consistieron exclusivamente en entrega de código y defensa técnica en vivo, sin un informe escrito –, este documento sí exige explicitar las correcciones realizadas a lo largo del semestre. El historial de Git del proyecto permite reconstruir esa evolución con precisión, sin depender de la memoria:

Mapeo de errores HTTP. Hasta antes de la corrección, cualquier recurso no encontrado (por ejemplo, un libro con un `id` inexistente) devolvía `500 Internal Server Error` en lugar de `404`, porque no existía un manejo de excepciones dedicado. Se creó una excepción `RecursoNoEncontradoException` y un `GlobalExceptionHandler` por servicio (reutilizando el que ya existía en catálogo en vez de duplicarlo), lo que también reactivó bloques `catch` (`FeignException.NotFound`) que hasta entonces eran código muerto.

HATEOAS. Inicialmente solo el microservicio de catálogo implementaba HATEOAS. Se extendió el mismo patrón (`EntityModel/CollectionModel` con enlaces *self*) a los nueve microservicios restantes.

Validaciones cruzadas vía Feign. Se agregaron validaciones adicionales entre servicios que aún no las tenían: `carrito` valida contra `clientes` y `catálogo`; `reseñas` valida contra `catálogo` y `clientes`; `pagos` y `envíos` validan contra `pedidos`.

Cobertura de pruebas. Se incorporó JaCoCo al POM raíz (antes no existía medición real de cobertura) y se agregó una clase de prueba `@WebMvcTest` por servicio, que verifica los códigos HTTP reales devueltos por cada endpoint. La cobertura de la lógica de negocio (controladores y servicios) es hoy de **93,6 %** (capítulo 7).

Relación JPA real entre entidades. Se agregó al microservicio de envíos una segunda entidad, `HistorialEstado`, en una relación real `@OneToMany/@ManyToOne` con `Envio`, que registra cada cambio de estado del envío. Se optó por este servicio en particular porque la relación es puramente aditiva: no modifica el contrato de ningún endpoint existente.

Datos de prueba con Datafaker. Se incorporó un `DataSeeder` por servicio (`CommandLineRunner`) que genera datos de ejemplo realistas la primera vez que cada tabla está vacía, reutilizando la propia capa de servicio (y por lo tanto las validaciones Feign existentes) en lugar de insertar filas directamente en el repositorio.

Validación con Postman. Se construyó una colección Postman que cubre CRUD y comunicación REST entre microservicios para los diez servicios, verificada en vivo contra el stack completo levantado en Docker antes de darla por válida (capítulos 5 y 6).

Autenticación en clientes. Se agregó JWT + BCrypt al microservicio de clientes, protegiendo únicamente las operaciones de escritura sobre el propio perfil (sección 3). Se optó por este alcance acotado en vez de proteger todo el microservicio, precisa-

mente porque **carrito** y **reseñas** dependen de poder leerlo vía Feign sin un token de usuario.

Reportes de ventas. Se agregaron reportes de ventas por sucursal y por autor en **pedidos** (sección 5), cerrando parcialmente la exclusión de reportes documentada originalmente en la sección 2.

5. Operaciones CRUD con JPA/ORM validadas con Postman

5.1. Persistencia con Spring Data JPA

Cada microservicio modela su dominio como una entidad JPA independiente, persistida en su propia base de datos mediante `JpaRepository`, sin necesidad de escribir SQL manual para las operaciones CRUD básicas. El siguiente ejemplo corresponde a la entidad `Libro` del microservicio de catálogo:

Listing 5.1: Libro.java – entidad JPA de catálogo

```
1 @Entity
2 @Table(name = "libros")
3 @Data
4 public class Libro {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String titulo;
10    private String autor;
11    private Double precio;
12 }
```

Listing 5.2: LibroRepository.java – repositorio

```
1 public interface LibroRepository extends JpaRepository<Libro, Long> {
2 }
```

La sola extensión de `JpaRepository<Libro, Long>` otorga las operaciones `save`, `findById`, `findAll` y `deleteById` sin implementación adicional. En los servicios donde la consulta lo requiere se agregan métodos derivados (por ejemplo, `findByLibroIdAndSucursal` en inventario, o `findByCodigoSeguimiento` en envíos), que Spring Data traduce automáticamente a JPQL a partir del nombre del método.

Sobre esta base, cada servicio expone las cuatro operaciones CRUD que correspondan a su dominio a través de endpoints REST. Por ejemplo, el microservicio de clientes expone las cuatro operaciones completas:

Operación	Método HTTP	Endpoint
Crear	POST	/api/v1/clientes
Leer (uno)	GET	/api/v1/clientes/{id}
Leer (todos)	GET	/api/v1/clientes
Actualizar	PUT	/api/v1/clientes/{id}
Eliminar	DELETE	/api/v1/clientes/{id}

Otros servicios exponen un subconjunto de estas operaciones cuando el dominio no requiere todas – por ejemplo, catálogo no expone DELETE porque un libro referenciado por pedidos o reseñas no debe poder eliminarse sin más.

5.2. Validación con la colección Postman

Para cumplir con el requisito de validar las operaciones CRUD con datos provenientes de Postman, se construyó una colección (`postman/BookPoint.postman_collection.json`) que cubre los diez microservicios, documentada en `postman/README.md`.

La colección está organizada en once carpetas ordenadas según la cadena real de dependencias entre servicios (catálogo → clientes → inventario → carrito → pedidos → pagos → envíos → reseñas → notificaciones → sucursales), de modo que puede ejecutarse de principio a fin con *Collection Runner*. Cada request de creación incluye un script de prueba que captura el `id` devuelto por la API y lo guarda en una variable de colección, reutilizada automáticamente por los siguientes requests – por ejemplo, el `id` del libro creado en la carpeta de catálogo se reutiliza al registrar stock en inventario, agregarlo al carrito y generar un pedido.

Listing 5.3: Script de test de Postman – captura automática de ID al crear un libro

```

1 pm.test('Status 201', () => pm.response.to.have.status(201));
2 const json = pm.response.json();
3 pm.collectionVariables.set('libroId', json.id);

```

Todas las requests de la colección fueron probadas en vivo contra los diez servicios levantados en Docker antes de considerarlas válidas – no se trata de una colección escrita en teoría, sino verificada extremo a extremo, incluyendo los datos generados automáticamente por Datafaker como insumo (sección 4.5).

5.3. Reportes de ventas

El caso pide que el perfil Jefe de Sucursal pueda "generar reportes de ventas por categoría, autor o sucursal". Se implementaron los dos primeros en el microservicio de pedidos, que ya concentra los datos de venta (`sucursal`, `libroId`, `total`):

- GET `/api/v1/pedidos/reportes/por-sucursal`: agregación directa sobre la propia base de datos de pedidos (GROUP BY `sucursal` vía una consulta JPQL con proyección de interfaz de Spring Data).
- GET `/api/v1/pedidos/reportes/por-autor`: como el autor no vive en la base de datos de pedidos sino en la de catálogo, no es posible resolverlo con un JOIN SQL.

Se agregan primero las ventas por `libroId`, y luego se enriquece cada grupo con el autor correspondiente vía Feign, reagrupando en memoria los libros que comparten autor.

El reporte por categoría se dejó fuera: exigiría agregar un campo de categoría o género al catálogo, un cambio de esquema en otro microservicio no motivado por ninguna otra necesidad actual del sistema.

Se verificó en vivo, contra el stack completo levantado en Docker con datos generados por Datafaker, que ambos reportes son consistentes entre sí: la suma total de ventas agrupada por sucursal coincide exactamente con la suma total agrupada por autor (\$796.892 en ambos casos sobre el mismo conjunto de pedidos), confirmando que ninguna venta se pierde ni se duplica al reagrupar.

6. Comunicación RESTful Entre Microservicios

6.1. Feign Client como mecanismo de comunicación

La comunicación entre microservicios se implementa de forma declarativa mediante **Spring Cloud OpenFeign**, en lugar de construir las llamadas HTTP a mano con **RestTemplate** o **WebClient**. Un **@FeignClient** se declara como una interfaz Java; Spring Cloud genera la implementación en tiempo de ejecución y resuelve la dirección real del servicio destino consultando Eureka – de modo que ningún microservicio necesita conocer el host o puerto de otro:

Listing 6.1: CatalogoClient.java (inventario) – Feign client

```
1 @FeignClient(name = "bookpoint-ms-catalogo")
2 public interface CatalogoClient {
3
4     @GetMapping("/api/v1/catalogo/{id}")
5     EntityModel<LibroRentDTO> obtenerLibroPorId(@PathVariable("id") Long id);
6 }
```

La Tabla 6.1 resume las dependencias reales entre microservicios, todas implementadas mediante este mecanismo.

Microservicio	Depende vía Feign de
Inventario	Catálogo
Carrito	Clientes, Catálogo
Pedidos	Catálogo, Inventario
Pagos	Pedidos
Envíos	Pedidos
Reseñas	Catálogo, Clientes

Cuadro 6.1: Dependencias reales entre microservicios vía Feign

Los microservicios de sucursales y notificaciones no consumen a ningún otro servicio: sus dominios son autocontenidos.

6.2. Manejo de errores en comunicación remota

Una llamada Feign puede fallar de dos maneras cualitativamente distintas, y el proyecto las distingue explícitamente en lugar de tratarlas de forma genérica:

1. **El recurso remoto no existe** (el servicio remoto respondió, pero con 404): se traduce a una excepción de negocio local (`RecursoNoEncontradoException`), que a su vez el `GlobalExceptionHandler` traduce a un 404 en la respuesta del servicio que hizo la llamada.
2. **No se pudo establecer comunicación** (el servicio remoto no responde, está caído, o Eureka aún no propagó su registro): se traduce a un error de servidor (500), porque no es un problema del dato solicitado sino de la infraestructura.

Listing 6.2: `InventarioServiceImpl.java` – distinción entre 404 remoto y error de red

```

1 try {
2     libroRemoto = catalogoClient.obtenerLibroPorId(inventarioDTO.getLibroId());
3 } catch (feign.FeignException.NotFound e) {
4     throw new RecursoNoEncontradoException(
5         "El libro con ID " + inventarioDTO.getLibroId()
6         + " no existe en el catalogo.");
7 } catch (Exception e) {
8     throw new RuntimeException(
9         "No se pudo conectar con el catalogo. Error de red: "
10        + e.getMessage());
11 }

```

Esta distinción no es cosmética: un cliente de la API que recibe un 404 sabe que el dato solicitado no existe y puede corregir su petición; uno que recibe un 500 sabe que debe reintentar más tarde, porque el problema es de disponibilidad, no de los datos enviados.

6.3. Validación con Postman

La colección Postman incluye una carpeta dedicada, *"Errores y validaciones cruzadas (404/400)"*, con casos negativos verificados en vivo contra el stack completo:

- Registrar stock de un libro inexistente en inventario → 404 (validación cruzada contra catálogo).
- Crear un pedido con un libro inexistente → 404 (validación cruzada contra catálogo).
- Crear un pedido con una cantidad mayor al stock disponible → 400 ("Stock insuficiente").
- Procesar un pago de un pedido inexistente → 404 (validación cruzada contra pedidos).

Cada uno de estos casos fue ejecutado contra los diez microservicios levantados en Docker (no simulado) antes de incorporarlo a la colección, incluyendo la verificación manual de que el mensaje de error y el código HTTP devueltos coincidieran exactamente con lo documentado en este informe.

7. Pruebas Unitarias

7.1. Herramientas utilizadas

Las pruebas automatizadas del proyecto utilizan:

- **JUnit 5** como framework de ejecución.
- **Mockito** para simular (*mock*) repositorios y clientes Feign en las pruebas de la capa de servicio, aislándola de la base de datos real y de los demás microservicios.
- **@WebMvcTest** + **@MockitoBean** de Spring Boot Test para las pruebas de la capa de controlador, que verifican el código HTTP real devuelto por cada endpoint (200, 201, 400, 404) sin levantar el contexto completo de Spring ni requerir base de datos.
- **JaCoCo** (`jacoco-maven-plugin`), integrado en el POM raíz, para medir la cobertura real de código ejercitada por las pruebas.

7.2. Estrategia de pruebas

Cada servicio cuenta con dos niveles de prueba:

1. **Pruebas de servicio** (`XServiceImplTest`): verifican la lógica de negocio con los repositorios y clientes Feign mockeados, siguiendo el patrón *Arrange-Act-Assert*. Cubren tanto el camino feliz como los casos de error (recurso no encontrado, argumento inválido).
2. **Pruebas de controlador** (`XControllerTest`): verifican que cada endpoint devuelva el código HTTP correcto y la forma esperada de respuesta (incluyendo los enlaces HATEOAS), con el servicio mockeado mediante `@MockitoBean`.

El siguiente ejemplo, tomado de las pruebas del microservicio de envíos, ilustra ambos elementos: la verificación del camino feliz y la del registro correcto en el historial de estados (la relación JPA agregada en la sección 4.5):

Listing 7.1: `EnvioServiceImplTest.java` – fragmento

```
1 @Test
2 void deberiaCrearEnvioExitosamente() {
3     Envio envio = new Envio();
4     envio.setPedidoId(1L);
5     when(pedidoClient.obtenerPedidoPorId(1L)).thenReturn(new PedidoRentDTO());
6     when(envioRepository.save(any(Envio.class)))
```

```

7      .thenAnswer(invocacion -> invocacion.getArgument(0));
8
9      Envio resultado = envioService.crearEnvio(envio);
10
11      assertEquals("PREPARACION", resultado.getEstado());
12      verify(envioRepository, times(1)).save(any(Envio.class));
13      verify(historialEstadoRepository, times(1)).save(any(HistorialEstado.class))
14      ;
15  }

```

7.3. Resultados de cobertura

La ejecución de `mvn test` sobre los doce módulos del proyecto arroja **168 pruebas** en total, todas exitosas. La Tabla 7.1 presenta la cobertura de líneas medida con JaCoCo, calculada de dos formas distintas:

- **Cobertura de lógica de negocio:** solo paquetes `controller`, `service` y `mapper` – el código que realmente contiene reglas de negocio y flujo de la aplicación.
- **Cobertura del módulo completo:** incluye además clases de configuración (`WebConfig`, `OpenApiConfig`), la clase principal de arranque de Spring Boot, y los `DataSeeder` de Datafaker.

Microservicio	Lógica de negocio	Módulo completo
Catálogo	100,0 %	71,9 %
Clientes	100,0 %	70,6 %
Inventario	100,0 %	53,3 %
Notificaciones	100,0 %	70,3 %
Sucursales	100,0 %	70,1 %
Pedidos	95,9 %	39,6 %
Envíos	93,8 %	57,8 %
Pagos	88,9 %	49,5 %
Carrito	81,0 %	45,4 %
Reseñas	84,3 %	46,5 %
Total	93,6 %	55,2 %

Cuadro 7.1: Cobertura de líneas por microservicio (JaCoCo)

La diferencia entre ambas columnas es intencional y se explica completamente: los `DataSeeder` agregados para cumplir el requisito de Datafaker (sección 4.5) son clases `CommandLineRunner` con lógica de reintento contra Eureka, verificadas mediante ejecución real en Docker contra el stack completo levantado – no mediante pruebas unitarias con mocks, ya que su propósito es precisamente ejercitar la comunicación real entre servicios al arrancar. Contarlas como "no cubiertas" en un reporte de JaCoCo orientado a pruebas unitarias no refleja que su comportamiento no haya sido validado, sino que se validó con una herramienta distinta (verificación en vivo) y no cuenta como cobertura sobre las clases de configuración (`WebConfig`, `OpenApiConfig`) tampoco aporta información relevante, dado que no

contienen lógica de negocio a probar. Por eso se reporta la cobertura de lógica de negocio como el indicador principal de calidad de las pruebas.

8. Documentación HATEOAS

8.1. Framework elegido: Spring HATEOAS

Se eligió **Spring HATEOAS** como framework para implementar el nivel de madurez de Richardson correspondiente a hipertexto (HATEOAS), por su integración nativa con Spring Boot: no requiere construir manualmente las URLs de los enlaces, sino que las deriva directamente de los métodos del propio controlador mediante `WebMvcLinkBuilder`, evitando que los enlaces queden desincronizados si cambia una ruta.

Listing 8.1: LibroController.java – respuesta con enlace HATEOAS

```
1 @GetMapping("/{id}")
2 public ResponseEntity<EntityModel<LibroDTO>> obtenerPorId(@PathVariable Long id)
3 {
4     Libro libro = libroService.obtenerPorId(id);
5     LibroDTO dto = libroMapper.toDTO(libro);
6
7     EntityModel<LibroDTO> model = EntityModel.of(dto);
8     model.add(linkTo(methodOn(LibroController.class)
9         .obtenerPorId(id)).withSelfRel());
10    model.add(linkTo(methodOn(LibroController.class)
11        .listar()).withRel("listar"));
12
13    return ResponseEntity.ok(model);
14 }
```

Cada respuesta de un recurso individual incluye al menos un enlace *self*, y en varios casos un enlace adicional hacia el recurso relacionado más natural (por ejemplo, de un libro hacia el listado completo, o de una reseña hacia el promedio de calificaciones del libro correspondiente).

8.2. Navegabilidad de la API

El siguiente ejemplo, tomado de una respuesta real capturada durante la verificación en Docker, muestra la forma exacta en que Spring HATEOAS serializa un recurso individual (formato HAL): los campos del recurso se aplanan al nivel superior del JSON, junto al objeto `_links`:

Listing 8.2: Respuesta real de POST `/api/v1/catalogo`

```
1 {
```

```
2  "_links": {
3    "self": { "href": "http://localhost:8081/api/v1/catalogo/3" }
4  },
5  "id": 3,
6  "titulo": "Cien Anos de Soledad",
7  "autor": "Gabriel Garcia Marquez",
8  "precio": 15990.0
9  }
```

Para una colección, en cambio, el formato HAL cambia de estructura: en lugar de un arreglo plano, la respuesta envuelve los elementos dentro de `_embedded`:

Listing 8.3: Respuesta real de un listado con HATEOAS

```
1  {
2    "_embedded": {
3      "historialEstadoList": [
4        { "id": 1, "estado": "PREPARACION", "envioId": 16},
5        { "id": 2, "estado": "EN_CAMINO", "envioId": 16}
6      ]
7    },
8    "_links": {
9      "self": { "href": "http://localhost:8088/api/v1/envios/16/historial" }
10   }
11 }
```

Esta diferencia estructural entre un recurso individual (aplanado) y una colección (`_embedded`) no es cosmética: un consumidor de la API que espere un arreglo plano de JSON (como un `List<X>` tipado en un cliente Feign) fallará al deserializar una respuesta HATEOAS de colección. Este problema se presentó realmente al extender HATEOAS al microservicio de inventario: el `InventarioClient` que consume pedidos esperaba `List<InventarioRentDTO>`, y tuvo que actualizarse a `CollectionModel<EntityModel<InventarioRentDTO>>` para reflejar la forma real de la respuesta:

Listing 8.4: `InventarioClient.java` (pedidos) – adaptado a la forma HATEOAS real

```
1  // inventario expone esta lista envuelta en HATEOAS (CollectionModel), no como
2  // un arreglo plano -- hay que declarar el tipo tal cual lo devuelve el
   productor.
3  @GetMapping("/api/v1/inventario/libro/{libroId}")
4  CollectionModel<EntityModel<InventarioRentDTO>> obtenerStockPorLibro(
5    @PathVariable("libroId") Long libroId);
```

Este detalle refuerza por qué la navegabilidad HATEOAS debe considerarse desde el diseño del contrato entre microservicios, y no agregarse como una capa cosmética al final del desarrollo.

9. Conclusión

El sistema desarrollado responde directamente a los tres síntomas identificados en el caso BookPoint Chile: la lentitud de procesos se aborda mediante microservicios independientes que escalan y se despliegan sin afectarse entre sí; el stock desactualizado entre sucursales se resuelve centralizando el inventario en un servicio único consultado por todos los canales; y las demoras de despacho se abordan con un microservicio de envíos dedicado, capaz de registrar y exponer el historial completo de estados de cada envío.

La arquitectura resultante – diez microservicios independientes, cada uno con su propia base de datos, comunicándose vía Feign y descubriéndose mediante Eureka detrás de un API Gateway – cubre en profundidad el flujo comercial completo del caso: explorar el catálogo, agregar productos al carrito, comprar, pagar, despachar y reseñar. Quedan deliberadamente fuera de alcance funciones administrativas de soporte (autenticación y roles, reportes de venta, promociones, transferencias entre sucursales), priorizando profundidad y calidad en el flujo principal por sobre cobertura superficial de la totalidad del caso.

Un aspecto que vale la pena destacar de este proceso es que las correcciones documentadas en la sección 4.5 no fueron anticipadas desde el diseño inicial, sino descubiertas y corregidas iterativamente a medida que se sometía el sistema a verificación real – probando cada flujo contra el stack completo levantado en Docker, en vez de asumir que el código funcionaba por haber compilado. Esa disciplina de verificación empírica es, en sí misma, una de las buenas prácticas que este informe buscó demostrar.